

```

20. ci = letter_to_index[ciphertext[i]]
21. ki = (ci - pi) % 26
22. recovered_key += index_to_letter[ki]
23. print("Recovered Key:", recovered_key)

```

Upon executing the program 'kpa28.sage', the output shown in Figure 1 is obtained. As can be verified, the secret key is successfully recovered.

```

# sage kpa28.sage
Ciphertext: KIPARTICMRVRYISVNIX
Recovered Key: DEEPU

```

Figure 1: Known-plaintext attack

Now, let us go through the code line by line to understand how it works. Lines 1 to 11 implement the Vigenère cipher encryption. Recall that this code is already known to us. Line 12 defines the plaintext message to be encrypted as 'HELLOOPENSOURCEFORYOU'. Line 13 defines the encryption key as 'DEEPU'. Line 14 calls the function `vigenere_encrypt()` (defined earlier) to encrypt the plaintext using the key 'DEEPU'. Line 15 prints the ciphertext generated from the previous step. Line 16 initialises an empty string named `recovered_key`. Line 17 defines the length of the original key (which is 5, since 'DEEPU' has 5 characters) and stores it in a variable named `key_length`. Recall that the key length in the Vigenère cipher can be determined using the Kasiski examination, as discussed in the article covering the Vigenère cipher. The loop in lines 18 to 22 recovers the original key using the known-plaintext attack method. The variable `pi` has the index of the i^{th} plaintext character (for example, 'H' $\rightarrow$ 7). The variable `ci` has the index of the i^{th} ciphertext character. The variable `ki` has the index of the i^{th} key character, recovered by reversing the encryption formula (Line 21). Finally, Line 23 prints the recovered key.

Now, consider the SageMath program named 'cpa29.sage' shown below, which implements a chosen-plaintext attack on the Vigenère cipher. To enable Vigenère cipher encryption, lines 1 to 11 of the program 'kpa28.sage' should be added at the beginning of the program 'cpa29.sage'.

```

chosen_plaintext = "AAAAA"
ciphertext = vigenere_encrypt(chosen_plaintext, secret_key)
print("Ciphertext received:", ciphertext)
recovered_key = ""
for i in range(len(chosen_plaintext)):
    pi = letter_to_index[chosen_plaintext[i]]
    ci = letter_to_index[ciphertext[i]]
    ki = (ci - pi) % 26
    recovered_key += index_to_letter[ki]
print("Recovered Key:", recovered_key)

```

Upon executing the program 'cpa29.sage', the output shown in Figure 2 is obtained. As can be verified, the secret key is successfully recovered.

```

# sage cpa29.sage
Ciphertext received: DEEPU
Recovered Key: DEEPU

```

Figure 2: Chosen-plaintext attack

Now, let us understand how the program works. The attacker selects the chosen plaintext 'AAAAA', which is then encrypted using the secret key via the function `vigenere_encrypt()`. Since the letter 'A' corresponds to index 0, each character in the resulting ciphertext directly reveals the shift applied at that position—effectively exposing the corresponding key character. As a result, the program successfully reconstructs the original key by leveraging the predictable behaviour of the Vigenère encryption when a known input is used.

Once we are familiar with the chosen-plaintext attack, it becomes easier to understand the chosen-ciphertext attack, as both involve controlled access to the encryption or decryption process to recover the key or plaintext. In a chosen-plaintext attack, submitting a plaintext like 'AAAAA' causes the ciphertext to directly reflect the key, since the shift applied to each 'A' corresponds to the key character. Similarly, in a chosen-ciphertext attack, submitting a ciphertext like 'AAAAA' and observing its decrypted plaintext allows the attacker to compute the key by reversing the shift. Although one starts with plaintext and the other with ciphertext, both attacks exploit the regular, repeating structure of the Vigenère cipher to uncover the key with minimal effort.

The examples discussed here are intentionally simple, but the same underlying principles can be applied to attack stronger, widely used ciphers such as 3DES, AES, and others. Moreover, beyond the attacks we have already mentioned, there are more advanced and potent techniques—such as the meet-in-the-middle attack, differential cryptanalysis, and linear cryptanalysis. The threat posed by all these attacks becomes significantly more serious with the computational speedup offered by quantum computers, potentially compromising encryption schemes once considered secure. Thus, quantum computers are expected to create a surge in demand for skilled computer professionals—similar to the Y2K era, but on a much larger and more complex scale.

Public key cryptography

Now, it is time to uncover how public-key cryptography (PKC) works. PKC is often seen as counter-intuitive because it openly shares what seems to be half of the key—the public key—yet still maintains strong security. Traditionally, secrecy in cryptography depended on keeping the entire key hidden, as seen in symmetric encryption. However, in public-key cryptography, one key (the public key) is made available to everyone, while